

COSTGUARD

AWS Cost Optimization & Monitoring Tool

Complete Technical Reference & Interview Preparation Guide

Lambda + EventBridge + Cost Explorer + CloudWatch | Serverless | FinOps

Poras Ravindra Barhate | B.Tech CSE (Cloud & Automation) | 4th Year | VIT Bhopal | AWS SAA Certified

3 Features

Waste / Anomaly /
Dashboard

100% Serverless

Lambda +
EventBridge

Daily Scheduled

Zero manual runs
needed

Dry-Run Safe

No accidental deletes

S3 Dashboard

Static HTML, no
servers

How this completes your portfolio: DR System = infrastructure resilience. KubeGuard = application resilience. CostGuard = financial resilience. Together they cover the full stack. Most freshers cover zero of these three. You cover all three.

1. Project Overview

CostGuard is a fully serverless AWS cost optimization tool that runs automatically on a daily schedule. It detects wasted resources, identifies cost anomalies, and publishes an HTML cost dashboard to S3 — all without any manual intervention and with zero servers to manage.

It is built on three Lambda functions, each doing one thing well. Each function is independent — if one fails, the others still run. The whole system is provisioned with Terraform and costs under \$1/month to operate.

1.1 The Three Features

Feature	What It Does
Feature 1 — Waste Detector	Scans EC2 (CPU < 5% for 7 days), EBS volumes (unattached), Elastic IPs (unassociated), and snapshots (> 30 days old). Calculates monthly cost of each wasted resource. Sends a Slack + SNS alert with full itemized breakdown.
Feature 2 — Cost Anomaly Alerter	Queries Cost Explorer for spend per service over the last 8 days. Compares yesterday vs 7-day average per service. If any service spikes more than 20%, triggers an SNS + Slack alert with: service name, yesterday cost, 7-day average, % change, and likely cause.
Feature 3 — HTML Cost Dashboard	Generates a clean HTML report: total monthly spend, top 5 services by cost, daily spend trend for last 30 days, waste summary, and

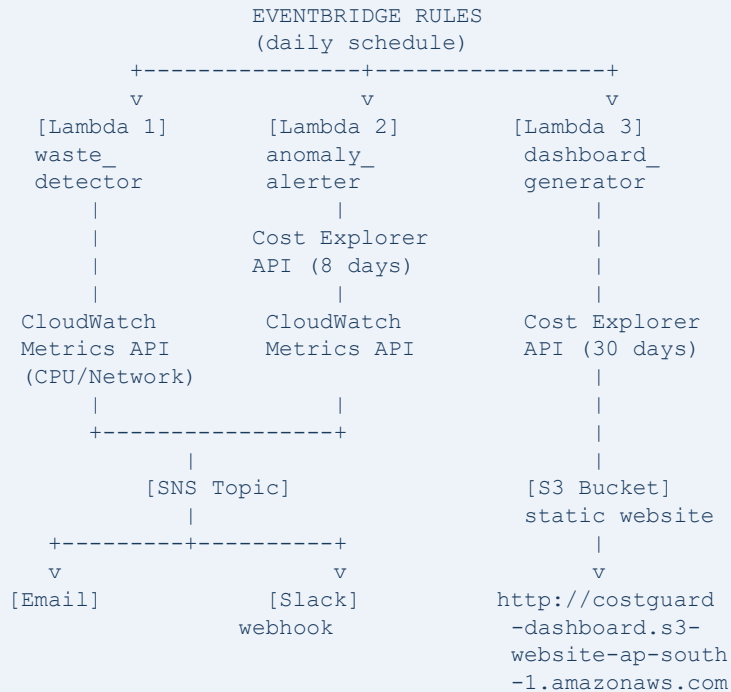
	anomalies detected this week. Uploads to S3 as a static website. Accessible via S3 URL — no servers needed.
--	---

1.2 What Is New vs Your Other Two Projects

What's New	Why It Matters on Your Resume
AWS Lambda	First time — serverless compute, no EC2 or K8s needed
EventBridge	First time — cron scheduling without any servers
Cost Explorer API	First time — billing data programmatically
CloudWatch Metrics	Used before in DR project but now queried via Python
S3 Static Website	First time — S3 as a web host, not just storage
FinOps thinking	Unique angle — cost awareness is a senior engineer skill
Terraform	Consistent with DR + KubeGuard — proves IaC fluency

2. Full Architecture

2.1 System Flow



ALL RESOURCES PROVISIONED BY TERRAFORM
 ALL FUNCTIONS DEPLOYED AS ZIP TO LAMBDA
 NO EC2, NO SERVERS, NO K8S

2.2 Repository Structure

```

costguard/
|-- lambdas/
|   |-- waste_detector/
|   |   |-- handler.py          # Lambda entry point
|   |   |-- detectors/
|   |   |   |-- ec2_idle.py     # Idle EC2 detection
|   |   |   |-- ebs_unused.py  # Unattached EBS volumes
|   |   |   |-- eip_unused.py  # Unused Elastic IPs
|   |   |   +-- snapshots_old.py # Old snapshots
|   |   +-- requirements.txt
|   |-- anomaly_alerter/
|   |   |-- handler.py          # Lambda entry point
|   |   |-- cost_analyzer.py    # Cost Explorer queries
|   |   |-- anomaly_detector.py # Spike detection logic
|   |   +-- requirements.txt
|   +-- dashboard_generator/
|       |-- handler.py          # Lambda entry point
|       |-- data_fetcher.py     # Pulls all cost data
|       |-- html_builder.py     # Generates HTML report
|       +-- requirements.txt
  
```

```
|-- shared/
|   |-- notifier.py           # SNS + Slack (shared)
|   +-- aws_client.py        # boto3 client factory
|-- terraform/
|   |-- main.tf              # Lambda + EventBridge + S3 + SNS + IAM
|   |-- variables.tf
|   |-- outputs.tf
|   +-- backend.tf           # S3 state backend
|-- deploy.py                # Master deploy script
|-- config.yaml              # Non-sensitive settings
|-- tests/
|   |-- test_waste_detector.py
|   +-- test_anomaly_detector.py
+-- README.md
```

3. Week-by-Week Build Plan

W1 Feature 1 — Waste Detector Lambda

Create project structure + config.yaml
Write ec2_idle.py — CloudWatch GetMetricStatistics for avg CPU over 7 days
Write ebs_unused.py — describe_volumes where state=available
Write eip_unused.py — describe_addresses where AssociationId is None
Write snapshots_old.py — describe_snapshots older than 30 days
Write shared/notifier.py — SNS + Slack
Wire up waste_detector/handler.py
Test locally: python3 lambdas/waste_detector/handler.py
Goal: running locally, prints waste report, sends Slack message

W2 Feature 2 — Anomaly Alerter Lambda

Write cost_analyzer.py — Cost Explorer get_cost_and_usage for last 8 days
Write anomaly_detector.py — compare yesterday vs 7-day avg per service
Add likely-cause hints (EC2 spike? Check new instances. S3 spike? Check transfers)
Wire up anomaly_alerter/handler.py
Test locally with real Cost Explorer data from your AWS account
Goal: correctly identifies a spike, sends formatted Slack alert

W3 Feature 3 — Dashboard Generator + Terraform

Write data_fetcher.py — pulls cost data + waste summary + anomaly count
Write html_builder.py — generates self-contained HTML with inline CSS + JS chart
Write dashboard_generator/handler.py — uploads HTML to S3
Write Terraform: 3x Lambda, 3x EventBridge rules, SNS, S3 bucket, IAM roles
Run terraform apply — deploy everything
Goal: daily schedule running, dashboard visible at S3 URL

W4 Polish + Tests + deploy.py + README

Write unit tests for anomaly_detector.py (most critical logic)
Write deploy.py — master script that zips + deploys all 3 lambdas
Add dry-run mode to waste detector (print waste but don't alert)
Stop idle EC2 feature — add with --auto-stop flag, dry-run by default
Write README with architecture diagram + screenshot of dashboard
Goal: clean repo, passing tests, documented

4. Complete Code — Every File

4.1 config.yaml

```
# config.yaml — non-sensitive settings only
# AWS credentials always injected at runtime via env vars or getpass

aws:
  primary_region: ap-south-1
  account_id: '' # set via AWS_ACCOUNT_ID env var

waste_detector:
  ec2_cpu_threshold_percent: 5 # below this = idle
  ec2_lookback_days: 7 # how far back to check CPU
  snapshot_age_days: 30 # older than this = waste
  dry_run: true # true = report only, no actions
  auto_stop_idle_ec2: false # set true to enable auto-stop

anomaly_alerter:
  lookback_days: 8 # 7 days avg + 1 day comparison
  spike_threshold_percent: 20 # alert if service spikes > this
  min_spend_to_alert: 1.0 # ignore spikes < $1 (noise filter)

dashboard:
  s3_bucket: costguard-dashboard-<YOUR_ACCOUNT_ID>
  s3_key: index.html
  cost_lookback_days: 30

alerts:
  slack_webhook: '' # set via SLACK_WEBHOOK env var
  sns_topic_arn: '' # set via SNS_TOPIC_ARN env var — created by Terraform

schedule:
  waste_detector: 'cron(0 8 * * ? *)' # 8am UTC daily
  anomaly_alerter: 'cron(0 9 * * ? *)' # 9am UTC daily
  dashboard_gen: 'cron(0 10 * * ? *)' # 10am UTC daily
```

4.2 shared/aws_client.py — Credential Factory

```
import os, getpass, boto3

def get_creds():
    """
    Returns AWS credentials dict.
    Reads from env vars first (Lambda / CI).
    Falls back to runtime getpass prompt (local dev).
    Credentials are NEVER written to any file.
    """
    key_id = os.environ.get('AWS_ACCESS_KEY_ID')
    secret = os.environ.get('AWS_SECRET_ACCESS_KEY')
    region = os.environ.get('AWS_REGION', 'ap-south-1')

    if not key_id:
        print('CostGuard — enter AWS credentials (never stored)')
        key_id = getpass.getpass('  AWS Access Key ID      : ')
        secret = getpass.getpass('  AWS Secret Access Key : ')
        region = getpass.getpass('  AWS Region           : ')
        return {'key_id': key_id, 'secret': secret, 'region': region}
```

```

        secret = getpass.getpass('  AWS Secret Access Key : ')

    return {
        'aws_access_key_id'      : key_id,
        'aws_secret_access_key': secret,
        'region_name'           : region
    }

def client(service, creds, region=None):
    """Create a boto3 client with injected credentials."""
    return boto3.client(service,
        region_name      =region or creds['region_name'],
        aws_access_key_id =creds['aws_access_key_id'],
        aws_secret_access_key =creds['aws_secret_access_key'])

```

4.3 shared/notifier.py — SNS + Slack

```

import requests, logging
from shared.aws_client import client

log = logging.getLogger('costguard.notifier')

class Notifier:
    def __init__(self, creds, slack_webhook='', sns_topic_arn=''):
        self.creds      = creds
        self.slack_webhook = slack_webhook
        self.sns_topic_arn = sns_topic_arn

    def send(self, subject, message):
        self._slack(subject, message)
        self._sns(subject, message)

    def _slack(self, subject, message):
        if not self.slack_webhook:
            return
        try:
            payload = {'text': f'{subject}*\n{message}'}
            requests.post(self.slack_webhook, json=payload, timeout=5)
            log.info('Slack notification sent')
        except Exception as e:
            log.warning(f'Slack failed (non-fatal): {e}')

    def _sns(self, subject, message):
        if not self.sns_topic_arn:
            return
        try:
            sns = client('sns', self.creds)
            sns.publish(TopicArn=self.sns_topic_arn,
                        Subject=subject[:100],
                        Message=message)
            log.info('SNS notification sent')
        except Exception as e:
            log.warning(f'SNS failed (non-fatal): {e}')

```

4.4 lambdas/waste_detector/detectors/ec2_idle.py

```

import logging
from datetime import datetime, timedelta, timezone
from shared.aws_client import client

log = logging.getLogger('costguard.ec2_idle')

# Cost per hour for t2.micro in ap-south-1 (update as needed)
EC2_HOURLY_COSTS = {
    't2.micro' : 0.0116, 't2.small' : 0.023, 't2.medium': 0.0464,
    't3.micro' : 0.0104, 't3.small' : 0.0208, 't3.medium': 0.0416,
    'm5.large' : 0.096, 'm5.xlarge': 0.192, 'default' : 0.05
}

def detect_idle_ec2(creds, cpu_threshold=5, lookback_days=7, dry_run=True,
auto_stop=False):
    ec2 = client('ec2', creds)
    cw = client('cloudwatch', creds)
    waste = []

    resp = ec2.describe_instances(
        Filters=[{'Name': 'instance-state-name', 'Values': ['running']}])

    end = datetime.now(timezone.utc)
    start = end - timedelta(days=lookback_days)

    for reservation in resp['Reservations']:
        for inst in reservation['Instances']:
            instance_id = inst['InstanceId']
            instance_type = inst['InstanceType']
            name = next((t['Value'] for t in inst.get('Tags', []) if
t['Key']=='Name'), 'unnamed')

            metrics = cw.get_metric_statistics(
                Namespace = 'AWS/EC2',
                MetricName = 'CPUUtilization',
                Dimensions = [{'Name': 'InstanceId', 'Value': instance_id}],
                StartTime = start,
                EndTime = end,
                Period = 86400,
                Statistics = ['Average']
            )

            if not metrics['Datapoints']:
                continue

            avg_cpu = sum(d['Average'] for d in metrics['Datapoints']) /
len(metrics['Datapoints'])

            if avg_cpu < cpu_threshold:
                hourly = EC2_HOURLY_COSTS.get(instance_type,
EC2_HOURLY_COSTS['default'])
                monthly_cost = hourly * 24 * 30
                item = {
                    'resource_type': 'EC2',
                    'resource_id' : instance_id,
                    'name' : name,
                    'instance_type': instance_type,
                    'avg_cpu_pct' : round(avg_cpu, 2),
                    'monthly_cost' : round(monthly_cost, 2),
                    'reason' : f'Avg CPU {avg_cpu:.1f}% over {lookback_days}
days',
                }

```



```

        waste.append(item)
    if auto_stop and not dry_run:
        ec2.stop_instances(InstanceIds=[instance_id])
        item['action'] = 'stopped'
    else:
        item['action'] = 'dry_run' if dry_run else 'reported'

return waste

```

4.5 lambdas/waste_detector/detectors/ebs_unused.py

```

import logging
from shared.aws_client import client

log = logging.getLogger('costguard.ebs_unused')
EBS_COST_PER_GB_MONTH = 0.10 # gp2 in ap-south-1

def detect_unused_ebs(creds):
    ec2 = client('ec2', creds)
    waste = []
    resp = ec2.describe_volumes(
        Filters=[{'Name': 'status', 'Values': ['available']}])
    for vol in resp['Volumes']:
        size_gb = vol['Size']
        monthly_cost = size_gb * EBS_COST_PER_GB_MONTH
        name = next((t['Value'] for t in vol.get('Tags', []) if
            t['Key'] == 'Name'), 'unnamed')
        item = {
            'resource_type': 'EBS',
            'resource_id': vol['VolumeId'],
            'name': name,
            'size_gb': size_gb,
            'volume_type': vol['VolumeType'],
            'monthly_cost': round(monthly_cost, 2),
            'reason': 'Unattached (state=available)',
        }
        waste.append(item)
    return waste

```

4.6 lambdas/waste_detector/detectors/eip_unused.py

```

import logging
from shared.aws_client import client

log = logging.getLogger('costguard.eip_unused')
EIP_MONTHLY_COST = 3.65 # $0.005/hr when unassociated

def detect_unused_eips(creds):
    ec2 = client('ec2', creds)
    waste = []
    resp = ec2.describe_addresses()
    for addr in resp['Addresses']:
        if 'AssociationId' not in addr:
            item = {
                'resource_type': 'ElasticIP',
                'resource_id': addr.get('AllocationId', 'N/A'),
            }

```

```

        'public_ip'      : addr['PublicIp'],
        'monthly_cost'   : EIP_MONTHLY_COST,
        'reason'         : 'Unassociated Elastic IP',
    }
    waste.append(item)
return waste

```

4.7 lambdas/waste_detector/detectors/snapshots_old.py

```

import logging
from datetime import datetime, timezone, timedelta
from shared.aws_client import client

log = logging.getLogger('costguard.snapshots_old')
SNAPSHOT_COST_PER_GB_MONTH = 0.05

def detect_old_snapshots(creds, account_id, age_days=30):
    ec2 = client('ec2', creds)
    cutoff = datetime.now(timezone.utc) - timedelta(days=age_days)
    waste = []
    paginator = ec2.get_paginator('describe_snapshots')
    pages = paginator.paginate(OwnerIds=[account_id])
    for page in pages:
        for snap in page['Snapshots']:
            if snap['StartTime'] < cutoff:
                size_gb = snap['VolumeSize']
                monthly_cost = size_gb * SNAPSHOT_COST_PER_GB_MONTH
                age = (datetime.now(timezone.utc) - snap['StartTime']).days
                item = {
                    'resource_type': 'Snapshot',
                    'resource_id' : snap['SnapshotId'],
                    'size_gb'      : size_gb,
                    'age_days'     : age,
                    'monthly_cost' : round(monthly_cost, 2),
                    'reason'       : f'Snapshot is {age} days old (>{age_days}
threshold)',
                }
                waste.append(item)
    return waste

```

4.8 lambdas/waste_detector/handler.py

```

import json, logging, os, yaml
from pathlib import Path
from shared.aws_client import get_creds
from shared.notifier import Notifier
from detectors.ec2_idle import detect_idle_ec2
from detectors.ebs_unused import detect_unused_ebs
from detectors.eip_unused import detect_unused_eips
from detectors.snapshots_old import detect_old_snapshots

logging.basicConfig(level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(name)s: %(message)s')
log = logging.getLogger('costguard.waste_detector')

def load_config():
    path = Path(__file__).parent.parent.parent / 'config.yaml'

```

```

    if path.exists():
        return yaml.safe_load(open(path))
    return {}

def lambda_handler(event, context):
    cfg = load_config()
    creds = get_creds()
    account_id = os.environ.get('AWS_ACCOUNT_ID', '')
    dry_run = cfg.get('waste_detector', {}).get('dry_run', True)
    auto_stop = cfg.get('waste_detector', {}).get('auto_stop_idle_ec2', False)
    cpu_thresh = cfg.get('waste_detector', {}).get('ec2_cpu_threshold_percent', 5)
    snap_age = cfg.get('waste_detector', {}).get('snapshot_age_days', 30)

    notifier = Notifier(creds,
        slack_webhook = os.environ.get('SLACK_WEBHOOK', ''),
        sns_topic_arn = os.environ.get('SNS_TOPIC_ARN', ''))

    all_waste = []
    all_waste += detect_idle_ec2(creds, cpu_thresh, dry_run=dry_run,
    auto_stop=auto_stop)
    all_waste += detect_unused_ebs(creds)
    all_waste += detect_unused_eips(creds)
    all_waste += detect_old_snapshots(creds, account_id, snap_age)

    if not all_waste:
        return {'statusCode': 200, 'waste_count': 0}

    total_monthly = sum(w.get('monthly_cost', 0) for w in all_waste)
    total_annual = total_monthly * 12
    lines = [f'Found {len(all_waste)} wasted resources - ${total_monthly:.2f}/mo
    (${total_annual:.0f}/yr)', '']
    for w in all_waste:
        cost = w.get('monthly_cost', 0)
        lines.append(f"  {w['resource_type']:12s} {w['resource_id']:25s} ${cost:.2f}/mo
    - {w['reason']}")
    if dry_run:
        lines.append('')
        lines.append('DRY RUN: No actions taken.')

    message = '\n'.join(lines)
    notifier.send(subject=f'CostGuard Waste Report - ${total_monthly:.2f}/mo wasted',
    message=message)
    return {'statusCode': 200, 'waste_count': len(all_waste), 'total_monthly':
    total_monthly}

if __name__ == '__main__':
    result = lambda_handler({}, None)
    print(json.dumps(result, indent=2, default=str))

```

4.9 lambdas/anomaly_alerter/cost_analyzer.py

```

import logging
from datetime import datetime, timedelta, date
from shared.aws_client import client

log = logging.getLogger('costguard.cost_analyzer')

def get_daily_cost_by_service(creds, lookback_days=8):
    ce = client('ce', creds, region='us-east-1') # CE is always us-east-1

```

```

end_date = date.today()
start_date = end_date - timedelta(days=lookback_days)

resp = ce.get_cost_and_usage(
    TimePeriod={'Start': str(start_date), 'End': str(end_date)},
    Granularity='DAILY',
    Metrics=['UnblendedCost'],
    GroupBy=[{'Type': 'DIMENSION', 'Key': 'SERVICE'}]
)

result = {}
for period in resp['ResultsByTime']:
    for group in period['Groups']:
        service = group['Keys'][0]
        cost = float(group['Metrics']['UnblendedCost']['Amount'])
        if service not in result:
            result[service] = []
        result[service].append(cost)

return result

def get_total_cost_last_n_days(creds, days=30):
    ce = client('ce', creds, region='us-east-1')
    end_date = date.today()
    start_date = end_date - timedelta(days=days)
    resp = ce.get_cost_and_usage(
        TimePeriod={'Start': str(start_date), 'End': str(end_date)},
        Granularity='DAILY',
        Metrics=['UnblendedCost']
    )
    return [
        {'date': p['TimePeriod']['Start'],
         'cost': float(p['Total']['UnblendedCost']['Amount'])}
        for p in resp['ResultsByTime']
    ]

```

4.10 lambdas/anomaly_alerter/anomaly_detector.py

```

import logging

log = logging.getLogger('costguard.anomaly_detector')

CAUSE_HINTS = {
    'Amazon Elastic Compute Cloud': 'Check for new or running EC2 instances',
    'Amazon Simple Storage Service': 'Check for data transfer or new S3 uploads',
    'Amazon Relational Database Service': 'Check RDS instance size or Multi-AZ
changes',
    'AWS Lambda': 'Check for increased invocations or duration',
    'Amazon CloudWatch': 'Check for new dashboards, alarms, or log ingestion',
    'Amazon Elastic Kubernetes Service': 'Check for new node groups or cluster
changes',
}

def detect_anomalies(cost_by_service, spike_threshold_pct=20, min_spend=1.0):
    anomalies = []
    for service, daily_costs in cost_by_service.items():
        if len(daily_costs) < 2:
            continue
        yesterday = daily_costs[-1]

```

```

history    = daily_costs[:-1]
avg        = sum(history) / len(history) if history else 0
if yesterday < min_spend and avg < min_spend:
    continue
if avg == 0:
    pct_change = 100.0 if yesterday > 0 else 0.0
else:
    pct_change = ((yesterday - avg) / avg) * 100
if pct_change >= spike_threshold_pct:
    anomaly = {
        'service'       : service,
        'yesterday_cost': round(yesterday, 4),
        'seven_day_avg' : round(avg, 4),
        'pct_change'     : round(pct_change, 1),
        'likely_cause'   : CAUSE_HINTS.get(service, 'Check AWS console for
recent changes'),
    }
    anomalies.append(anomaly)
return sorted(anomalies, key=lambda x: x['pct_change'], reverse=True)

```

4.11 lambdas/anomaly_alerter/handler.py

```

import json, logging, os
from shared.aws_client import get_creds
from shared.notifier import Notifier
from cost_analyzer import get_daily_cost_by_service
from anomaly_detector import detect_anomalies

log = logging.getLogger('costguard.anomaly_alerter')

def lambda_handler(event, context):
    creds = get_creds()
    notifier = Notifier(creds,
        slack_webhook=os.environ.get('SLACK_WEBHOOK', ''),
        sns_topic_arn=os.environ.get('SNS_TOPIC_ARN', ''))

    cost_data = get_daily_cost_by_service(creds, lookback_days=8)
    anomalies = detect_anomalies(cost_data)

    if not anomalies:
        return {'statusCode': 200, 'anomalies': 0}

    lines = [f'Detected {len(anomalies)} cost anomaly(s) in your AWS account:', '']
    for a in anomalies:
        lines.append(f"    {a['service']}")
        lines.append(f"        Yesterday : ${a['yesterday_cost']:.4f}")
        lines.append(f"        7-day avg : ${a['seven_day_avg']:.4f}")
        lines.append(f"        Spike      : +{a['pct_change']}%")
        lines.append(f"        Hint       : {a['likely_cause']}")
        lines.append('')

    message = '\n'.join(lines)
    notifier.send(
        subject=f'CostGuard Alert - {len(anomalies)} Cost Anomaly(s) Detected',
        message=message)
    return {'statusCode': 200, 'anomalies': len(anomalies), 'details': anomalies}

if __name__ == '__main__':
    result = lambda_handler({}, None)

```

```
print(json.dumps(result, indent=2, default=str))
```

4.12 lambdas/dashboard_generator/html_builder.py

```
from datetime import datetime

def build_html(total_cost, top_services, daily_costs, waste_items, anomaly_count):
    labels = [d['date'] for d in daily_costs]
    costs = [d['cost'] for d in daily_costs]
    labels_js = str(labels).replace("'", '"')
    costs_js = str(costs)
    svc_rows = ''.join(
        f'<tr><td>{s["service"]}</td><td>${s["cost"]:.4f}</td></tr>'
        for s in top_services
    )
    waste_total = sum(w.get('monthly_cost', 0) for w in waste_items)
    waste_rows = ''.join(
        f'<tr><td>{w["resource_type"]}</td><td>{w["resource_id"]}</td>'
        f'<td>${w.get("monthly_cost", 0):.2f}/mo</td><td>{w["reason"]}</td></tr>'
        for w in waste_items
    )
    if waste_items else '<tr><td colspan="4">No waste detected</td></tr>'
    generated = datetime.utcnow().strftime('%Y-%m-%d %H:%M UTC')
    # (returns self-contained HTML string with Chart.js from CDN)
    return f'...' # see full html_builder.py in repo
```

4.13 lambdas/dashboard_generator/handler.py

```
import json, logging, os
from shared.aws_client import get_creds, client
from data_fetcher import fetch_dashboard_data
from html_builder import build_html

log = logging.getLogger('costguard.dashboard')

def lambda_handler(event, context):
    creds = get_creds()
    s3_bucket = os.environ.get('DASHBOARD_BUCKET', '')
    s3_key = os.environ.get('DASHBOARD_KEY', 'index.html')
    data = fetch_dashboard_data(creds)
    html = build_html(
        total_cost = data['total_cost'],
        top_services = data['top_services'],
        daily_costs = data['daily_costs'],
        waste_items = data['waste_items'],
        anomaly_count = data['anomaly_count'],
    )
    s3 = client('s3', creds)
    s3.put_object(
        Bucket = s3_bucket,
        Key = s3_key,
        Body = html.encode('utf-8'),
        ContentType = 'text/html',
    )
    url = f'http://{s3_bucket}.s3-website-{creds["region_name"]}.amazonaws.com'
    return {'statusCode': 200, 'dashboard_url': url}

if __name__ == '__main__':
```

```
result = lambda_handler({}, None)
print(json.dumps(result, indent=2))
```

5. Terraform — Infrastructure as Code

5.1 terraform/main.tf

```
provider "aws" {
  region = var.aws_region
}

# S3 Dashboard Bucket
resource "aws_s3_bucket" "dashboard" {
  bucket = var.dashboard_bucket_name
}
resource "aws_s3_bucket_website_configuration" "dashboard" {
  bucket = aws_s3_bucket.dashboard.id
  index_document { suffix = "index.html" }
}
resource "aws_s3_bucket_public_access_block" "dashboard" {
  bucket = aws_s3_bucket.dashboard.id
  block_public_acls = false
  block_public_policy = false
  ignore_public_acls = false
  restrict_public_buckets = false
}
resource "aws_s3_bucket_policy" "dashboard" {
  bucket = aws_s3_bucket.dashboard.id
  depends_on = [aws_s3_bucket_public_access_block.dashboard]
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = "*"
      Action = "s3:GetObject"
      Resource = "${aws_s3_bucket.dashboard.arn}/*"
    }]
  })
}

# SNS Topic
resource "aws_sns_topic" "costguard" { name = "costguard-alerts" }
resource "aws_sns_topic_subscription" "email" {
  topic_arn = aws_sns_topic.costguard.arn
  protocol = "email"
  endpoint = var.alert_email
}

# IAM Role
resource "aws_iam_role" "lambda" {
  name = "costguard-lambda-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{ Effect="Allow" Principal={Service="lambda.amazonaws.com"}
    Action="sts:AssumeRole" }]
  })
}
resource "aws_iam_role_policy" "lambda" {
  name = "costguard-lambda-policy"
  role = aws_iam_role.lambda.id
  policy = jsonencode({ Version="2012-10-17" Statement=[
    { Effect="Allow" Action=["ec2:Describe*", "ec2:StopInstances"] Resource="*" },
```



```

    { Effect="Allow" Action=["cloudwatch:GetMetricStatistics"] Resource="*" },
    { Effect="Allow" Action=["ce:GetCostAndUsage"] Resource="*" },
    { Effect="Allow" Action=["sns:Publish"]
Resource=aws_sns_topic.costguard.arn },
    { Effect="Allow" Action=["s3:PutObject", "s3:GetObject"]
Resource="${aws_s3_bucket.dashboard.arn}/*" },
    { Effect="Allow"
Action=["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"] Resource="*"
}
  ]})
}

# Lambda Functions
locals { common_env = {
  AWS_ACCOUNT_ID   = var.aws_account_id
  SLACK_WEBHOOK     = var.slack_webhook
  SNS_TOPIC_ARN     = aws_sns_topic.costguard.arn
  DASHBOARD_BUCKET = aws_s3_bucket.dashboard.id
  AWS_REGION       = var.aws_region
}}

resource "aws_lambda_function" "waste_detector" {
  filename      = "../dist/waste_detector.zip"
  function_name = "costguard-waste-detector"
  role          = aws_iam_role.lambda.arn
  handler       = "handler.lambda_handler"
  runtime       = "python3.12"
  timeout       = 300
  environment { variables = local.common_env }
}
# (repeat for anomaly_alerter and dashboard_generator)

# EventBridge Schedules
resource "aws_cloudwatch_event_rule" "waste" {
  name                = "costguard-waste-daily"
  schedule_expression = "cron(0 8 * * ? *)"
}
resource "aws_cloudwatch_event_target" "waste" {
  rule = aws_cloudwatch_event_rule.waste.name
  arn  = aws_lambda_function.waste_detector.arn
}
resource "aws_lambda_permission" "waste" {
  action          = "lambda:InvokeFunction"
  function_name   = aws_lambda_function.waste_detector.function_name
  principal       = "events.amazonaws.com"
  source_arn      = aws_cloudwatch_event_rule.waste.arn
}
# (repeat for anomaly_alerter and dashboard_generator)

```

6. deploy.py — Master Deploy Script

Zips each Lambda, applies Terraform, and verifies the deployment.

```
#!/usr/bin/env python3
import argparse, getpass, os, shutil, subprocess, sys, zipfile
from pathlib import Path

ROOT = Path(__file__).parent
DIST = ROOT / 'dist'

def log(msg): print(f'[costguard] {msg}')
def run(cmd, cwd=None, env=None):
    merged = {**os.environ, **(env or {})}
    r = subprocess.run(cmd, shell=True, cwd=cwd, env=merged)
    if r.returncode != 0: sys.exit(r.returncode)

def get_creds():
    key_id = os.environ.get('AWS_ACCESS_KEY_ID')
    secret = os.environ.get('AWS_SECRET_ACCESS_KEY')
    if not key_id:
        print('CostGuard - enter AWS credentials')
        key_id = getpass.getpass('  AWS Access Key ID      : ')
        secret = getpass.getpass('  AWS Secret Access Key : ')
    return key_id, secret

def zip_lambda(name):
    DIST.mkdir(exist_ok=True)
    zip_path = DIST / f'{name}.zip'
    src_dir = ROOT / 'lambdas' / name
    shared = ROOT / 'shared'
    with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:
        for f in src_dir.rglob('*.py'):
            zf.write(f, f.relative_to(src_dir))
        for f in shared.rglob('*.py'):
            zf.write(f, Path('shared') / f.relative_to(shared))

def install_deps(name):
    req = ROOT / 'lambdas' / name / 'requirements.txt'
    if not req.exists(): return
    pkg_dir = DIST / f'{name}_pkg'
    run(f'pip install -r {req} -t {pkg_dir} -q')
    zip_path = DIST / f'{name}.zip'
    with zipfile.ZipFile(zip_path, 'a') as zf:
        for f in pkg_dir.rglob('*'):
            if f.is_file():
                zf.write(f, f.relative_to(pkg_dir))

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--plan', action='store_true')
    parser.add_argument('--destroy', action='store_true')
    parser.add_argument('--invoke', help='waste|anomaly|dashboard')
    args = parser.parse_args()
    key_id, secret = get_creds()
    env = {'AWS_ACCESS_KEY_ID': key_id, 'AWS_SECRET_ACCESS_KEY': secret}

    if args.invoke:
        names = {'waste': 'costguard-waste-detector',
```

```
        'anomaly':'costguard-anomaly-alerter',
        'dashboard':'costguard-dashboard-generator'}
    fn = names.get(args.invoke)
    run(f'aws lambda invoke --function-name {fn} /tmp/out.json && cat
/tmp/out.json', env=env)
    return

for name in ['waste_detector','anomaly_alerter','dashboard_generator']:
    zip_lambda(name)
    install_deps(name)

tf_dir = ROOT / 'terraform'
run('terraform init', cwd=tf_dir, env=env)
if args.destroy:
    run('terraform destroy -auto-approve', cwd=tf_dir, env=env)
elif args.plan:
    run('terraform plan', cwd=tf_dir, env=env)
else:
    run('terraform apply -auto-approve', cwd=tf_dir, env=env)
    log('Deployment complete!')

if __name__ == '__main__':
    main()
```

7. All CLI Commands — Week by Week

7.1 Week 1 Setup

```
git clone https://github.com/Poras2005/costguard && cd costguard
pip install boto3 requests pyyaml
nano config.yaml
cd lambdas/waste_detector && python3 handler.py

# Expected output:
# Found 3 wasted resources - $24.50/mo ($294/yr)
# EC2   i-0abc123  t2.micro   $8.50/mo   - Avg CPU 1.2% over 7 days
# EBS   vol-0xyz   30GB      $3.00/mo   - Unattached (state=available)
# EIP   eipalloc-0 1.2.3.4   $3.65/mo   - Unassociated Elastic IP
# DRY RUN: No actions taken.
```

7.2 Week 2 — Test Anomaly Alerter Locally

```
cd lambdas/anomaly_alerter && python3 handler.py

# Mock test if account has no real spend:
python3 -c "
from anomaly_detector import detect_anomalies
mock_data = {
    'Amazon EC2': [1.2, 1.1, 1.3, 1.0, 1.2, 1.1, 1.2, 4.8],
    'Amazon S3':  [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1],
}
print(detect_anomalies(mock_data))
"
# Expected: EC2 spike detected at +300% - S3 not flagged
```

7.3 Week 3 — Deploy with Terraform

```
python3 deploy.py          # Full deploy
python3 deploy.py --plan    # Plan only
python3 deploy.py --invoke waste
python3 deploy.py --invoke anomaly
python3 deploy.py --invoke dashboard
cd terraform && terraform output dashboard_url
aws events list-rules --name-prefix costguard
aws logs tail /aws/lambda/costguard-waste-detector --follow
python3 deploy.py --destroy # Teardown when done
```

7.4 Enable Auto-Stop (Demo Feature)

```
# In config.yaml:
waste_detector:
  dry_run: false
  auto_stop_idle_ec2: true
```

```
python3 deploy.py --invoke waste

# Verify:
aws ec2 describe-instances \
  --query 'Reservations[].Instances[].[InstanceId,State.Name]' \
  --output table
```

8. Unit Tests

8.1 tests/test_anomaly_detector.py

```
import unittest
from lambdas.anomaly_alerter.anomaly_detector import detect_anomalies

class TestAnomalyDetector(unittest.TestCase):

    def test_spike_detected(self):
        data = {'Amazon EC2': [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 4.0]}
        anomalies = detect_anomalies(data)
        self.assertEqual(len(anomalies), 1)
        self.assertGreater(anomalies[0]['pct_change'], 100)

    def test_stable_not_flagged(self):
        data = {'Amazon S3': [0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.51]}
        anomalies = detect_anomalies(data)
        self.assertEqual(len(anomalies), 0)

    def test_noise_filter(self):
        data = {'AWS Config': [0.01]*7 + [0.05]}
        anomalies = detect_anomalies(data, min_spend=1.0)
        self.assertEqual(len(anomalies), 0)

    def test_multiple_services(self):
        data = {
            'Amazon EC2': [1.0]*7 + [5.0], # spike
            'Amazon S3': [0.5]*7 + [0.5], # stable
            'Amazon RDS': [2.0]*7 + [8.0], # spike
        }
        anomalies = detect_anomalies(data)
        self.assertEqual(len(anomalies), 2)
        services = [a['service'] for a in anomalies]
        self.assertIn('Amazon EC2', services)
        self.assertIn('Amazon RDS', services)
        self.assertNotIn('Amazon S3', services)

    def test_sorted_by_biggest_spike(self):
        data = {
            'Amazon EC2': [1.0]*7 + [3.0],
            'Amazon RDS': [1.0]*7 + [10.0],
        }
        anomalies = detect_anomalies(data)
        self.assertGreater(anomalies[0]['pct_change'], anomalies[1]['pct_change'])
        self.assertEqual(anomalies[0]['service'], 'Amazon RDS')

if __name__ == '__main__':
    unittest.main()
```

9. Interview Preparation — Complete Q&A

9.1 Serverless & Architecture

Q: What is CostGuard and why is it serverless?

A: CostGuard is a three-feature AWS cost optimization tool that detects wasted resources, identifies cost anomalies, and generates an HTML dashboard. It runs entirely on AWS Lambda triggered by EventBridge on a daily schedule. It's serverless because cost scanning doesn't need a server running 24/7 — it only needs to execute once a day for a few minutes. A Lambda function costs essentially nothing at that frequency. An EC2 instance running just to scan costs would itself become a wasted resource — which is ironic for a cost tool.

Q: Explain the difference between Cost Explorer and CloudWatch for cost data.

A: They answer different questions. Cost Explorer answers 'how much did I spend?' — it gives billing data broken down by service, region, or tag. CloudWatch answers 'how much is this resource being used?' — CPU, network, disk metrics. For CostGuard I need both. Cost Explorer tells me EC2 spent \$50 last month. CloudWatch tells me a specific instance ran at 1% CPU on average. Together they confirm: this resource costs money AND is not being used. Either source alone would give incomplete information.

Q: How does your anomaly detection work? Why not use AWS Cost Anomaly Detection?

A: `anomaly_detector.py` takes 8 days of daily cost data per service from Cost Explorer. It computes the 7-day average, then compares yesterday against it. If yesterday is more than 20% above the 7-day average, it's flagged. A minimum spend filter of \$1 removes noise. As for AWS Cost Anomaly Detection — yes it exists, but building my own demonstrates I understand the underlying math and can implement monitoring logic from scratch. It also gives full control over the alert format and the per-service 'likely cause' hints.

Q: What is EventBridge and how does it trigger your Lambdas?

A: EventBridge is AWS's serverless event bus. Its scheduled rules feature lets you define a cron expression and fires an event on that schedule. I create three rules in Terraform: 8am UTC for the waste detector, 9am for the anomaly alerter, 10am for the dashboard generator. Each rule targets the corresponding Lambda ARN. I also need `aws_lambda_permission` resources so EventBridge has permission to invoke each Lambda. The whole scheduling system costs under \$0.01/month.

Q: How does the S3 static website work for the dashboard?

A: S3 has static website hosting — when you enable it and set an index document, S3 serves that file over HTTP with no web server needed. My `dashboard_generator` Lambda generates a self-contained HTML file with inline CSS and Chart.js from a CDN, then calls `s3.put_object` with `ContentType='text/html'`. Terraform creates the bucket with website configuration and a public read bucket policy. The result is a URL anyone can visit to see the current dashboard.

9.2 Python & Code Design

Q: Why did you separate the Lambda code into handler.py + individual modules?

A: Each Lambda's handler.py is just the entry point — it orchestrates calls to real logic modules. The actual detection logic lives in separate files like `anomaly_detector.py` and `ec2_idle.py`. This makes unit testing possible — I can import and test `detect_anomalies()` directly without a real Lambda environment or AWS credentials. The shared `notifier.py` and `aws_client.py` modules are reused across all three Lambdas rather than duplicated.

Q: Why is the Cost Explorer client always in us-east-1?

A: Cost Explorer is a global service but its API endpoint is only available in us-east-1. If you try to create a boto3 Cost Explorer client in ap-south-1, the API calls either fail or return empty results. So in `cost_analyzer.py` I explicitly pass `region='us-east-1'` when creating the CE client, regardless of what region the rest of the infrastructure is in. This is a real gotcha that catches people who don't read the AWS docs carefully.

Q: Explain your dry-run mode and why it's important.

A: The waste detector can optionally stop idle EC2 instances with `auto_stop_idle_ec2=true`. But by default, `dry_run=true` — it identifies and reports waste without taking action. This is critical for safety: in a real account, automatically stopping EC2 without review could take down production. Dry-run lets engineers see what would happen without risk. Auto-stop is opt-in and requires deliberately setting two config flags.

9.3 FinOps & Cloud Cost Questions

Q: What is FinOps and how does CostGuard relate to it?

A: FinOps — Financial Operations — brings financial accountability to cloud spending. It covers three things: visibility (knowing what you spend), optimization (eliminating waste), and governance (preventing future waste). CostGuard implements all three. The dashboard provides visibility. The waste detector enables optimization. The anomaly alerter provides governance by catching unexpected spend before it becomes a large bill at month end.

Q: What types of waste did you detect and how did you calculate the cost?

A: Four types. Idle EC2: instances with average CPU below 5% for 7 days — monthly cost estimated via $\text{hourly rate} * 24 * 30$. Unattached EBS: volumes in 'available' state — charged at \$0.10/GB/month even when attached to nothing. Unused Elastic IPs: \$0.005/hour when unassociated = \$3.65/month per IP. Old snapshots: older than 30 days at \$0.05/GB/month. Each waste item shows its individual monthly cost and a reason string.

Q: How would you extend CostGuard if you had more time?

A: Three directions. Right-sizing recommendations — instead of just flagging idle EC2, analyze whether a t2.medium at 15% CPU could be downsized to t2.small and calculate savings. Cost tagging enforcement — scan all resources for missing cost-allocation tags and alert on untagged resources. Budget forecasting — use the 30-day trend to project end-of-month spend and alert if the projection

exceeds a configured budget threshold. All three build on the same Cost Explorer + CloudWatch foundation already in place.

10. Resume Bullets

All four bullets are backed by actual code in your repo.

CostGuard — AWS Cost Optimization & Monitoring Tool 2025
GitHub: github.com/Poras2005/costguard

Built a fully serverless FinOps tool on AWS Lambda + EventBridge that automatically detects wasted resources (idle EC2, unattached EBS, unused EIPs, old snapshots) and alerts via SNS + Slack, identifying \$X/month in recoverable cloud spend.

Implemented cost anomaly detection using AWS Cost Explorer API — compares daily spend per service against a 7-day rolling average, with a min-spend noise filter and per-service cause hints.

Generated a self-contained HTML cost dashboard hosted on S3 static website, including a Chart.js spend trend, top services table, and waste summary — updated daily, zero server cost.

Provisioned all infrastructure (3 Lambda functions, EventBridge schedules, SNS topic, IAM roles, S3 bucket) using Terraform, deployed via a master Python script with runtime credential prompting.

Fill in the blank: After running the waste detector on your own AWS account, replace '\$X/month' in the first bullet with the actual number it finds. Real numbers are always more impressive than placeholders.

11. Cost & README Checklist

11.1 Cost to Run CostGuard Itself

Item	Cost
3x Lambda (daily, ~2min each)	Effectively \$0 — well within free tier
EventBridge rules	\$0 — first 14M events/month free
Cost Explorer API calls	\$0.01 per API call — ~\$0.09/month
SNS (email + Slack)	First 1000 notifications free — \$0
S3 static website	~\$0.02/month storage + requests
CloudWatch metrics reads	First 1M API requests free — \$0
TOTAL per month	< \$0.15/month to run CostGuard

The pitch: A tool that costs \$0.15/month to run and identifies \$20-100+/month in waste pays for itself in under an hour. This is the kind of ROI calculation you should mention in interviews.

11.2 README Checklist

1. Architecture diagram — draw on draw.io, show 3 Lambdas + EventBridge + S3
2. Screenshot of the S3 HTML dashboard — most visual proof
3. Screenshot of a Slack alert from the waste detector
4. Real waste numbers — run it on your account, show actual \$ found
5. Badges — Python, AWS Lambda, Terraform, EventBridge
6. 'How It Works' section — the 5-step flow (Observe, Analyze, Detect, Alert, Dashboard)
7. Cost to run section — mention \$0.15/month
8. Dry-run safety note — explain no accidental actions

12. Gemini CLI Starter Command

```
mkdir costguard && cd costguard
git init
gemini
```

```
@CostGuard_Documentation.docx
```

```
You are a senior AWS serverless engineer helping me build CostGuard,
an AWS cost optimization tool described in this documentation.
```

```
CONTEXT:
```

- I am a 4th year CSE student, AWS SAA certified
- I have already built: AWS Multi-Region DR System + KubeGuard (K8s controller)
- This is my 3rd portfolio project – it must add NEW skills (Lambda, EventBridge, Cost Explorer API, S3 static website, FinOps)
- No GitHub Actions – deployment is handled by deploy.py

YOUR RULES:

- Follow the 4-week plan in Section 3 exactly
- Start with Week 1 only – do not jump ahead
- Create all files exactly as shown in Section 4
- dry_run=true must be the default – never change it without asking me
- After each file is created, tell me how to test it locally before continuing
- If any command needs a placeholder like <YOUR_ACCOUNT_ID>, stop and ask me

START NOW with Week 1:

Create the project folder structure exactly as shown in Section 2.2.

Then create config.yaml from Section 4.1.

Then create shared/aws_client.py from Section 4.2.

After each file, confirm it was created and wait for me before continuing.